

CSCI 4170 - Project in AI and ML

Homework 4 - Summer 2025

Haruto Suzuki

Due date: 11:59 pm, July. 9th

Task 1 (50 points): In this task, you will pick a dataset (time-series or any other form of sequential data) and an associated problem that can be solved via sequence models. You must describe why you need sequence models to solve this problem. Include a link to the dataset source. Next, you should pick an RNN framework that you would use to solve this problem (This framework can be in TensorFlow, PyTorch or any other Python Package).

Part 1 (10 points): Implement your RNN either using an existing framework OR you can implement your own RNN cell structure. In either case, describe the structure of your RNN and the activation functions you are using for each time step and in the output layer. Define a metric you will use to measure the performance of your model (NOTE: Performance should be measured both for the validation set and the test set).

A.

I implemented a simple RNN using PyTorch's `nn.RNN` module to predict the next day's closing price of Microsoft stock based on the previous 60 days of OHLCV data. The RNN consists of one hidden layer with 50 hidden units and uses the tanh activation function at each time step. I chose tanh because it outputs values in the range from -1 to 1 , which helps stabilize the hidden state by preventing exploding values and allows the model to capture both positive and negative temporal patterns in the data. This is important for time series like stock prices, which naturally fluctuate over time.

The output at the final time step is passed through a fully connected linear layer with no activation function. This is because the task is regression, which involves predicting a continuous numeric value such as the closing price. Applying an activation function like sigmoid or tanh would restrict the output range and limit the model's ability to produce realistic predictions. A linear output layer allows the model to produce any real number that fits the trend in the training data.

To evaluate the model, I use Root Mean Squared Error (RMSE), as it directly measures the average prediction error in the same units as the target variable. RMSE is computed separately for both the validation set and the test set to assess the model's performance and ability to generalize to unseen data.

Part 2 (30 points): Update your network from part 1 with first an LSTM and then a GRU based cell structure (You can treat both as 2 separate implementations). Re-do the training and performance evaluation. What are the major differences you notice? Why do you think those differences exist between the 3 implementations (basic RNN, LSTM and GRU)?

Note: In part 1 and 2, you must perform sufficient data-visualization, pre-processing and/or feature-engineering if needed. The overall performance visualization of the loss function should also be provided.

A.

We updated the basic RNN model with LSTM and GRU implementations to explore their effect on performance and temporal modeling capabilities. All models were trained under identical conditions using the same dataset and hyperparameters to ensure a fair comparison.

We observed that the RNN sometimes produces sharper fluctuations and roughly traces the shape of the actual curve, but it consistently suffers from vertical displacement, often predicting values below the true range. While the RNN is able to follow the general trend, it lacks the responsiveness to adapt to rapid changes in the data. Its predictions tend to be unstable and sensitive to small variations in training.

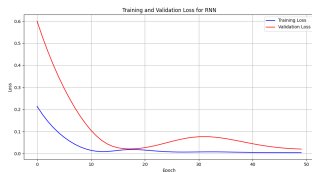
In contrast, the LSTM produces smoother outputs and more effectively captures both short-term and long-term dependencies. It responds more accurately to sudden spikes and dips in the series, resulting in better alignment with major trends compared to the RNN. However, like the RNN, LSTM occasionally exhibits vertical misalignment, indicating a systematic prediction shift.

The GRU exhibits characteristics that lie between the RNN and LSTM. It generates smoother predictions than the RNN but is slightly less aggressive than the LSTM in reacting to sharp changes. GRU achieves a balance between sensitivity and smoothness, making it an attractive alternative to LSTM, particularly when training time or architectural simplicity is a priority.

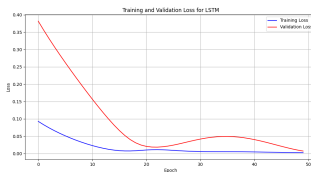
In terms of training dynamics, the loss curves behave similarly across all three models. Minor differences in output and convergence patterns may occur across different runs, indicating some variability due to initialization or training stochasticity.

The differences between the basic RNN, LSTM, and GRU arise primarily from their internal architectures and how they handle sequential information. The basic RNN lacks mechanisms to retain long-term dependencies, making it prone to vanishing gradient problems, which limits its ability to model longer trends accurately. It often underfits the data or becomes unstable, leading to outputs that may follow the shape of the actual values but with noticeable shifts. LSTM improves upon this by introducing a memory cell and gating mechanisms (input, forget, and output gates) that control the flow of information, allowing the model to remember relevant past inputs over long sequences. This results in better tracking of both slow and rapid changes in time series data. GRU simplifies the LSTM architecture by combining the gates into fewer components (reset and update gates), achieving

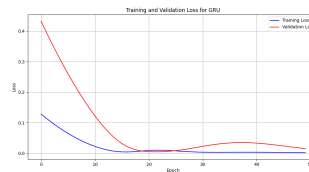
a similar ability to manage long-term dependencies while training faster and requiring fewer parameters. As a result, GRU often performs comparably to LSTM but may exhibit slightly smoother or less aggressive predictions.



(a) RNN Loss Curve



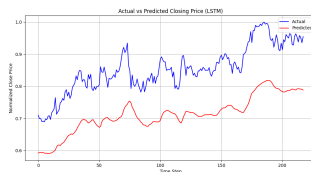
(b) LSTM Loss Curve



(c) GRU Loss Curve



(d) RNN Predictions



(e) LSTM Predictions



(f) GRU Predictions

Figure 1: Top: Training and validation loss for RNN, LSTM, and GRU. Bottom: Actual vs Predicted normalized closing prices for each model.

Part 3 (10 points): Can you use the traditional feed-forward network to solve the same problem. Why or why not? (Hint: Can time series data be converted to usual features that can be used as input to a feed-forward network?)

Yes, a traditional feed-forward network can be used to solve the same problem, but it requires careful preprocessing. Since feed-forward networks do not have a notion of temporal ordering or memory, the time-series data must be explicitly converted into fixed-size input vectors. This is typically done by using a sliding window approach, where the past n days of OHLCV data are flattened into a single feature vector that serves as the input, and the next day's closing price is used as the target. While this allows the network to learn from past values, it does not capture sequential dependencies as effectively as recurrent models like RNNs, LSTMs, or GRUs. Therefore, although technically feasible, feed-forward networks generally underperform on time-series forecasting tasks that involve long-term temporal patterns.

Task 2 (50 points):

Part 1: Implementing Word Embeddings (10 points)

- Use a pre-trained word embedding model (Word2Vec, GloVe, FastText, or BERT embeddings).

We will be using BERT embeddings.

- Provide a comparative discussion on why you chose this embedding over others.

We use BERT embeddings from the `bert-base-uncased` model via Hugging Face's `transformers` library. Compared to traditional models like Word2Vec, GloVe, and FastText, BERT produces context-aware embeddings by processing input through a bidirectional transformer. This means that a word's embedding depends on its surrounding words, allowing BERT to better capture nuances in meaning such as polysemy (e.g., "bank" as a financial institution versus "bank" of a river). While models like Word2Vec and FastText are faster and suitable for general similarity tasks, they produce static word vectors regardless of context. We chose BERT to take advantage of its deep contextual understanding and its robustness in handling diverse language inputs.

- Load embeddings efficiently (either from pre-trained vectors or using an NLP library like Gensim, SpaCy, or Hugging Face).

We load BERT efficiently using the `transformers` and `torch` libraries.

- Allow dynamic user input of two words and output their respective embeddings.

Our implementation allows users to input two words, which are then tokenized and passed through BERT to obtain their embeddings.

- Handle cases where a word is out of vocabulary (OOV) and suggest ways to approximate its embedding.

For out-of-vocabulary (OOV) words, BERT automatically breaks them into subword tokens using WordPiece. If a full word is not present in the BERT vocabulary, we compute the embedding by averaging its subword embeddings. This offers a reasonable approximation of the full word's semantics.

Part 2: Cosine Similarity Computation (20 points)

- Implement a function that computes the cosine similarity between two-word embeddings.

We implemented a cosine similarity function using PyTorch to compute similarity between word embeddings. Given two embeddings $\vec{a}$ and $\vec{b}$, cosine similarity is defined as:

$$\text{cosine_similarity}(\vec{a}, \vec{b}) = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \|\vec{b}\|}$$

- Explain why cosine similarity is useful in word embedding space.

Cosine similarity is useful in the word embedding space because it captures the angular difference between vectors, regardless of their magnitude. This is important because two words can be semantically similar even if their embedding magnitudes differ. Cosine similarity helps focus on the orientation of the word vectors, which reflects their semantic relationship in high-dimensional space.

- Allow batch processing, where users can input multiple word pairs for simultaneous similarity computation.

We support batch input of multiple word pairs. Each pair is tokenized, embedded using BERT, and their cosine similarity is computed. This enables efficient similarity analysis for large sets of words. As for this assignment, we will hard code the pair of words.

- Visualization Requirement: Create a 2D or 3D scatter plot (e.g., using PCA or t-SNE) to visually show how similar and dissimilar words cluster together in the embedding space.

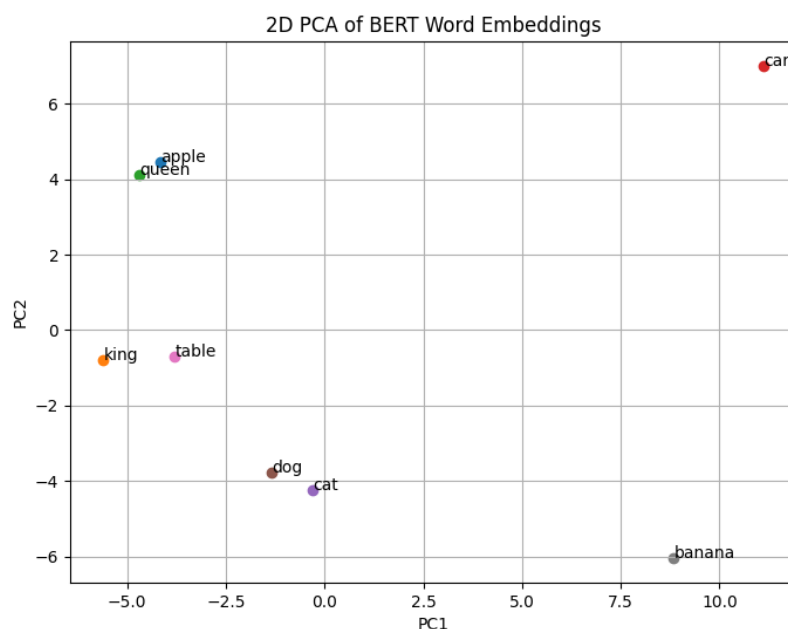


Figure 2: 2D PCA visualization of BERT embeddings for selected words. Clearly, similar words appear close together, such as “cat” and “dog”. In contrast, dissimilar words like “car” and “banana” are far apart in the embedding space. The words “king” and “queen” are related but slightly separated, which may reflect subtle differences such as gender or contextual usage.

Part 3: Designing a Novel Dissimilarity Metric (20 points)

- Define a custom dissimilarity score that goes beyond cosine similarity. Possible approaches include:
 - Euclidean distance (How far apart words are in vector space).
 - Word entropy-based dissimilarity (How uncommon two words are relative to each other in corpora).
 - Semantic contrast measure (Using external knowledge bases like WordNet).
- Either design your own metric or cite an existing one from literature (provide a proper reference). Explain why your metric captures novelty/diversity better than cosine similarity alone.
- Allow users to toggle between different similarity/dissimilarity measures via function parameters.
- Visualization Requirement:
 - Plot the ranking of words based on their similarity/dissimilarity to a given word (e.g., how words like "cat" rank against "dog," "lion," and "table" using different metrics).
 - Use a heatmap to demonstrate and compare similarity and dissimilarity across multiple (any number of your choice) word pairs.

Manhattan distance captures novelty/diversity better than cosine similarity because it considers both direction and magnitude of semantic differences, while cosine similarity only measures angles and ignores vector magnitudes entirely. This means Manhattan distance can distinguish between words with similar directions but different semantic intensities (like "like" vs "love"), whereas cosine similarity treats them as nearly identical. Additionally, Manhattan distance maintains better discriminative power in high-dimensional spaces where cosine similarity suffers from the curse of dimensionality. The linear sensitivity of Manhattan distance preserves subtle differences across all embedding dimensions that often represent genuine semantic novelty. RetryClaude can make mistakes. Please double-check responses.

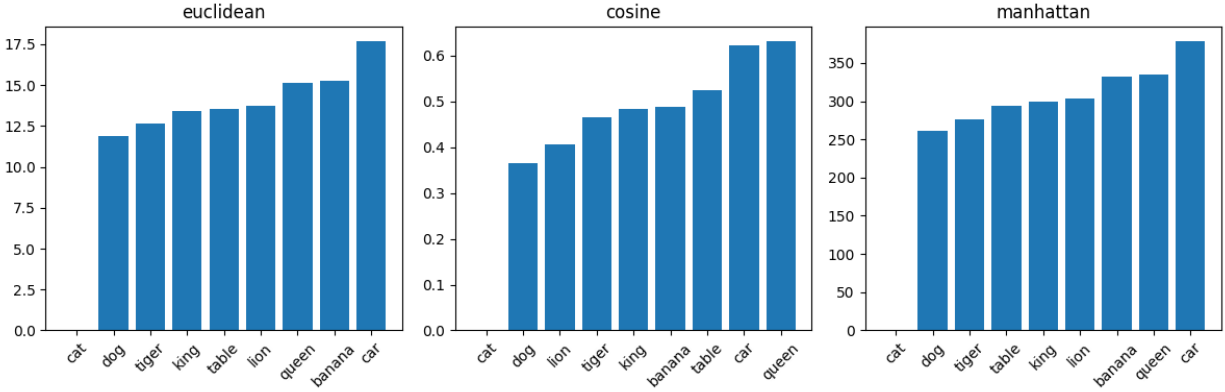


Figure 3: Bar chart comparing how different dissimilarity metrics rank various words based on their distance from the target word “cat”. Euclidean distance, cosine dissimilarity, and Manhattan distance yield slightly different rankings depending on how they interpret similarity in the embedding space. Cosine emphasizes angular alignment, while Euclidean and Manhattan capture absolute differences.

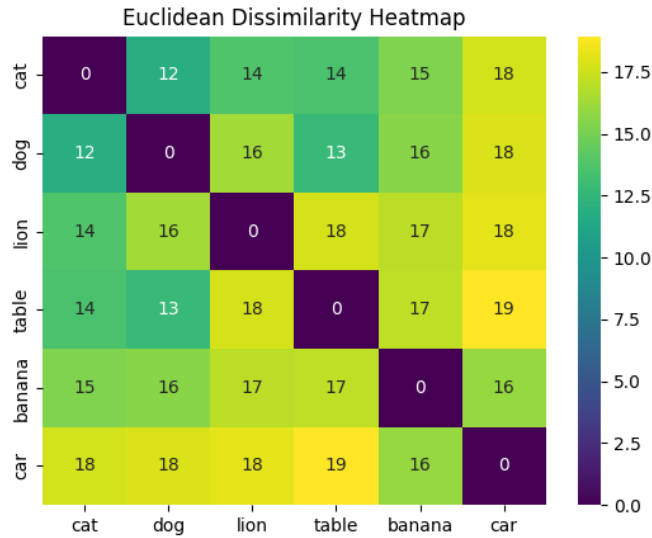


Figure 4: Euclidean distance heatmap of BERT word embeddings. Words with similar meanings, such as “cat” and “dog” or “king” and “queen”, appear closer together (darker colors), while dissimilar words like “car” and “banana” show higher distances (brighter colors).

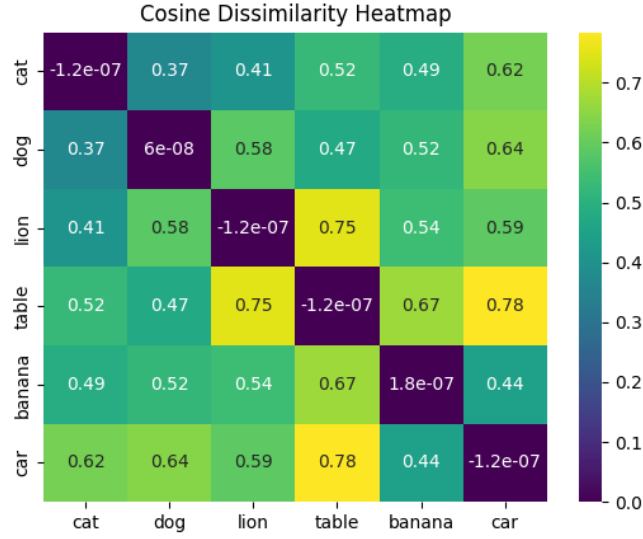


Figure 5: Cosine dissimilarity heatmap of BERT word embeddings. Cosine dissimilarity reflects angular separation between vectors, highlighting semantic similarity independent of vector magnitude. Words within similar domains are clustered together.

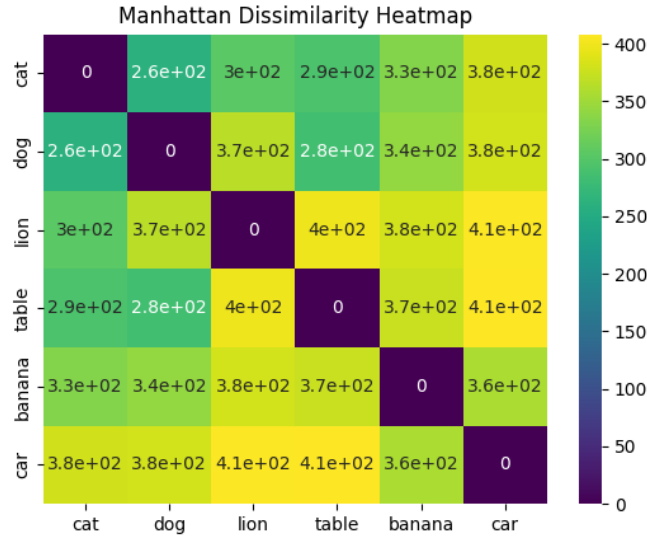


Figure 6: Manhattan (L1) distance heatmap of BERT word embeddings. This metric sums absolute differences across dimensions and often exaggerates small differences between similar words, leading to higher contrast in the heatmap.